

Git + GitHub

Savoir où l'on en est, avant de toucher à quoi que ce soit.

Un kit très court pour travailler proprement et faire grandir un portfolio GitHub lisible.

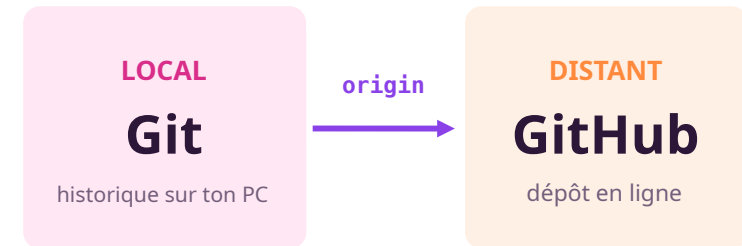
1 · Observer

2 · Comparer

3 · Agir

Avant un pull, un push ou un commit, vérifie simplement ce que Git voit.

LE MODÈLE MENTAL



branche = ta ligne de travail

Le réflexe : observer → comparer → agir.

Le check de 30 secondes avant de commencer

Cinq commandes suffisent pour répondre aux questions essentielles.

```
pwd  
git rev-parse --show-toplevel  
git branch --show-current  
git remote -v  
git status
```

01 Je suis dans le bon dossier ?

02 Quelle branche est active ?

03 À quel dépôt GitHub suis-je relié-e ?

04 Ai-je des changements non enregistrés ?

Règle : ne lance pas pull / push si tu ne comprends pas encore ce que montre status.

Local et GitHub sont-ils synchronisés ?

On met d'abord à jour notre vue du dépôt distant. Ensuite seulement, on compare.

PowerShell

```
git fetch origin
$branch = git branch --show-current
git status -sb

git rev-parse HEAD
git rev-parse "origin/$branch"

git diff --name-status "origin/$branch..HEAD"
git status --short
```

1

fetch

Actualise ce que ton ordinateur sait de GitHub. Aucun fichier local n'est fusionné.

2

HEAD vs origin

Compare les deux identifiants de commit. Identiques = même commit.

3

diff + status

Sépare les différences déjà committées des modifications encore dans ton dossier.

Astuce : `git status -sb` montre souvent directement « ahead » ou « behind ».

Lire le diagnostic sans paniquer

Le résultat te dit quelle est la prochaine action raisonnable.

ALIGNÉ

Même commit + status vide

Ton dossier local et GitHub sont alignés.

Tu peux travailler.

AHEAD

Tu as des commits locaux

GitHub ne les a pas encore reçus.

git push

BEHIND

GitHub a de nouveaux commits

Si ton dossier est propre, récupère-les sans créer de merge automatique.

git pull --ff-only

MODIFIÉ

status liste des fichiers

Tu as du travail non commité. Regarde-le avant toute synchro.

git diff

Cas plus complexe : « ahead » ET « behind » → inspecte avant de fusionner.

git log --oneline --decorate --graph --all -8

Le workflow quotidien sûr

Une routine courte, répétable, et facile à expliquer.

AVANT DE TRAVAILLER

```
git fetch origin
git status -sb

# si « behind » ET dossier propre
git pull --ff-only
```

- ✓ Tu sais exactement sur quelle base tu commences.
- ✓ Tu évites les merges surprises.

APRÈS TON TRAVAIL

```
git status
git diff
git add <fichier>
git diff --staged
git commit -m "Décris le changement"
git push
```

Regarde → sélectionne → vérifie → commit → push

Ton dépôt fait déjà partie du portfolio

Un bon projet doit être compréhensible par quelqu'un qui arrive dessus aujourd'hui.

À MONTRER

- ✓ **README clair : objectif, contexte, résultat**
- ✓ 1 capture ou démo visible rapidement
- ✓ Stack et instructions minimales
- ✓ Commits qui racontent des étapes compréhensibles
- ✓ Nom de dépôt propre + description GitHub utile

Ne pousse jamais : .env · clés API · mots de passe · données privées

SOS · DIAGNOSTIC À COPIER

Quand tu ne sais plus où tu en es, envoie les sorties de ces commandes :

```
git rev-parse --show-toplevel
git remote -v
git branch --show-current
git fetch origin
git status -sb
git log --oneline --decorate -5
```

Cela donne au mentor le contexte sans deviner.